DEPARTAMENTO DE
INGENIERÍA
INFORMÁTICA

Plataforma de arriendo de alojamientos turísticos

Taller de Bases de Datos · Grupo 4 · Laboratorio 1

- Diego Cáceres
- Dania Garrido
- Diego Lefiman
- Isidora Toledo
- Matias Urbano



Índice

¿Qué veremos?

Introducción y avance	El problema, los roles y lo que ya construimos
Arquitectura y demo	Capas del sistema y demostración
Fragmentos de código	Procedimientos, triggers, vista materializada e índices
Equipo y conclusiones	Aporte de cada integrante y lo que aprendimos

Introducción

¿De qué trata el proyecto?

Creamos un "marketplace" (estilo Airbnb) donde los dueños publican sus propiedades y los viajeros las reservan.

EL PROBLEMA

Asegurar que la información no choque como que dos personas reserven el mismo alojamiento en las mismas fechas, o que se cobre mal una estadía.

01

Huésped

Busca, reserva, paga y reseña sus estadías

02

Anfitrión

Publica alojamientos y define tarifas y políticas

03

Administrador

Modera publicaciones y reseñas; revisa los reportes

Descripción avance de proyecto

Las 10 tareas del laboratorio asignadas

ÁREA	TAREAS
API y seguridad	✓ Tarea 1 · CRUD de las entidades ✓ Tarea 2 · JWT y roles con Keycloak
Procedimientos	✓ Tarea 3 · Confirmar reserva (procedimiento 1) ✓ Tarea 4 · Cancelar con política de reembolso (procedimiento 2)
Triggers	✓ Tarea 5 · Validar reserva (traslape, capacidad, estadía mínima) ✓ Tarea 6 · Auditar cambios de precio y política
Vista e índices	✓ Tarea 7 · Vista materializada de ocupación e ingresos ✓ Tarea 8 · Índices (fechas con GIST, alojamiento y ciudad)
Endpoints	✓ Tarea 9 · Buscar alojamientos disponibles ✓ Tarea 10 · Promedio e historial de reseñas (solo API)

Arquitectura del sistema

Cómo se conectan el frontend, la API en Spring Boot y PostgreSQL

01

Keycloak

Login y JWT

Autentica al usuario y emite el JWT con sus roles

→

02

Frontend

React + Vite

Envía el JWT en cada petición a la API con axios

→

03

API REST

Spring Boot

Valida token y rol; Controller, Service y Repository

→

04

PostgreSQL

Datos y reglas

Ejecuta procedimientos y triggers y guarda los datos

Login en Keycloak → JWT al frontend → la API valida el token y consulta PostgreSQL

Funcionalidades

01

Login

Keycloak + JWT

Entramos como huésped y se muestra el token

02

Reserva

Rol huésped

El procedimiento crea reserva, pago y calendario

03

Evitar reservas dobles

Trigger en PostgreSQL

Otra reserva en las mismas fechas es rechazada

04

Cancelación

Política de reembolso

Se cancela y se registra el reembolso

Demostración

Algunos casos de uso

CASO 1 · RESERVA INVÁLIDA

Rechazo por trigger

Intentamos reservar con:

Más huéspedes que la capacidad

Menos noches que la estadía mínima

Fechas que se traslapan

La API responde 400 con el mensaje

CASO 2 · CANCELACIÓN

Reembolso por política

Según los días de anticipación:

Flexible: 100% desde 1 día

Moderada: 100% desde 5 días, 50% desde 2

Estricta: 100% desde 14 días, 50% desde 7

Pago negativo REEMBOLSO y fechas liberadas

Procedimientos Almacenados

PROCEDIMIENTO 1 · CONFIRMAR RESERVA

01

Calcular el total según la tarifa de cada noche

→

02

Registrar el pago

→

03

Bloquear las fechas en el calendario

→

04

Crear la reserva

PROCEDIMIENTO 2 · CANCELAR RESERVA

01

Aplicar la política de cancelación del alojamiento

→

02

Calcular el reembolso

→

03

Registrar la devolución

→

04

Liberar las fechas del calendario

Procedimiento almacenado 1

sp_confirmar_reserva: bloquea, verifica y guarda

PASO 1

Bloquear y verificar fechas

```
CREATE OR REPLACE PROCEDURE sp_confirmar_reserva(
    p_id_usuario INT,
    p_id_alojamiento INT,
    p_fecha_inicio DATE,
    p_fecha_fin DATE,
    p_n_huespedes INT,
    p_tipo_pago VARCHAR,
    INOUT p_id_reserva INT DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_fechas_ocupadas INT;
    v_total NUMERIC := 0;
    v_precio_noche NUMERIC;
    v_fecha_actual DATE;
BEGIN
    -- EVITAR DUPLICADOS EN CONCURRENCIA
    PERFORM id_alojamiento
    FROM alojamiento
    WHERE id_alojamiento = p_id_alojamiento
    FOR UPDATE;

    -- VALIDAR DISPONIBILIDAD
    SELECT COUNT(*) INTO v_fechas_ocupadas
    FROM calendario c
    JOIN reserva r ON c.id_reserva = r.id_reserva
    WHERE r.id_alojamiento = p_id_alojamiento
      AND c.fecha >= p_fecha_inicio
      AND c.fecha < p_fecha_fin;

    IF v_fechas_ocupadas > 0 THEN
        RAISE EXCEPTION 'El alojamiento no se encuentra disponible en una de las fechas seleccionadas';
    END IF;
```

PASO 2

Calcular, guardar y pagar

```
    -- CALCULAR EL TOTAL SEGUN LA TARIFA DE TEMPORADA DE CADA NOCHE
    v_fecha_actual := p_fecha_inicio;
    WHILE v_fecha_actual < p_fecha_fin LOOP
        SELECT precio INTO v_precio_noche
        FROM tarifa
        WHERE id_alojamiento = p_id_alojamiento
          AND v_fecha_actual BETWEEN fecha_inicio AND fecha_fin
        LIMIT 1;

        IF v_precio_noche IS NULL THEN
            SELECT precio_noche INTO v_precio_noche
            FROM alojamiento
            WHERE id_alojamiento = p_id_alojamiento;
        END IF;

        v_total := v_total + v_precio_noche;
        v_fecha_actual := v_fecha_actual + INTERVAL '1 day';
    END LOOP;

    -- CREAR RESERVA Y REGISTRAR EL PAGO
    INSERT INTO reserva (
        id_usuario, id_alojamiento, n_huespedes,
        fecha_inicio, fecha_fin, estado_reserva, precio_total
    )
    VALUES (
        p_id_usuario, p_id_alojamiento, p_n_huespedes,
        p_fecha_inicio, p_fecha_fin, 'CONFIRMADA', v_total
    )
    RETURNING id_reserva INTO p_id_reserva;

    INSERT INTO pago (id_reserva, monto_pago, fecha_pago, tipo_pago)
    VALUES (p_id_reserva, v_total, NOW(), p_tipo_pago);

    -- BLOQUEAR FECHAS EN EL CALENDARIO
    v_fecha_actual := p_fecha_inicio;
    WHILE v_fecha_actual < p_fecha_fin LOOP
        INSERT INTO calendario(id_reserva, fecha)
        VALUES (p_id_reserva, v_fecha_actual);
        v_fecha_actual := v_fecha_actual + INTERVAL '1 day';
    END LOOP;
END;
$$^;
```

Procedimiento almacenado 2

cancelarReserva: calcula el reembolso y libera las fechas

PASO 1

Leer y validar la reserva

```
CREATE OR REPLACE PROCEDURE cancelarReserva(
    p_id_reserva INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_actual VARCHAR;
    v_politica_cancelacion VARCHAR;
    v_fecha_inicio DATE;
    v_total_pagado INT;
    v_porcentaje_reembolso NUMERIC(5, 2);
    v_monto_reembolso INT;
    v_dias_anticipacion INTEGER;
    v_id_alojamiento INT;
BEGIN
    -- Datos de la reserva y del alojamiento
    SELECT r.estado_reserva, r.fecha_inicio, r.precio_total,
           r.id_alojamiento, a.politica_cancel
    INTO v_estado_actual, v_fecha_inicio, v_total_pagado,
         v_id_alojamiento, v_politica_cancelacion
    FROM reserva r
    JOIN alojamiento a ON r.id_alojamiento = a.id_alojamiento
    WHERE r.id_reserva = p_id_reserva;

    -- Validaciones
    IF v_estado_actual IS NULL THEN
        RAISE EXCEPTION 'La reserva con ID % no existe', p_id_reserva;
    END IF;
    IF v_estado_actual != 'CONFIRMADA' THEN
        RAISE EXCEPTION 'Solo se puede cancelar reservas con estado CONFIRMADA';
    END IF;

    -- Días restantes
    v_dias_anticipacion := v_fecha_inicio - CURRENT_DATE;
```

PASO 2

Reembolso, pago y calendario

```
    -- Porcentaje de reembolso según la política
    IF v_politica_cancelacion = 'flexible' THEN
        IF v_dias_anticipacion >= 1 THEN
            v_porcentaje_reembolso := 1.00;
        ELSE
            v_porcentaje_reembolso := 0.00;
        END IF;
    ELSIF v_politica_cancelacion = 'moderada' THEN
        IF v_dias_anticipacion >= 5 THEN
            v_porcentaje_reembolso := 1.00;
        ELSIF v_dias_anticipacion >= 2 THEN
            v_porcentaje_reembolso := 0.50;
        ELSE
            v_porcentaje_reembolso := 0.00;
        END IF;
    ELSIF v_politica_cancelacion = 'estricta' THEN
        IF v_dias_anticipacion >= 14 THEN
            v_porcentaje_reembolso := 1.00;
        ELSIF v_dias_anticipacion >= 7 THEN
            v_porcentaje_reembolso := 0.50;
        ELSE
            v_porcentaje_reembolso := 0.00;
        END IF;
    ELSE
        RAISE EXCEPTION 'Política de cancelación desconocida: %', v_politica_cancelacion;
    END IF;

    v_monto_reembolso := (v_total_pagado * v_porcentaje_reembolso)::INT;

    UPDATE reserva
    SET estado_reserva = 'CANCELADA'
    WHERE id_reserva = p_id_reserva;

    -- Devolución (monto negativo en pago)
    IF v_monto_reembolso > 0 THEN
        INSERT INTO pago (id_reserva, monto_pago, fecha_pago, tipo_pago)
        VALUES (p_id_reserva, -v_monto_reembolso, NOW(), 'REEMBOLSO');
    END IF;

    -- Liberar las fechas del calendario
    DELETE FROM calendario
    WHERE id_reserva = p_id_reserva;
END;
$$^;
```

Llamados de procedimientos almacenados

CALL

Repository/ReservaRepository.java

confirmarReserva

```java
// Confirmar reserva
public Integer confirmarReserva(Integer idUsuario, Integer idAlojamiento,
                                LocalDate fechaInicio, LocalDate fechaFin,
                                Integer nHuespedes, String tipoPago) {
String sql = """
    CALL sp_confirmar_reserva(
        :idUsuario, :idAlojamiento,
        CAST(:fechaInicio AS DATE), CAST(:fechaFin AS DATE),
        :nHuespedes, :tipoPago, NULL)
""";

// El parámetro INOUT p_id_reserva vuelve como una fila con una columna
return jdbcClient.sql(sql)
        .param(name: "idUsuario", idUsuario)
        .param(name: "idAlojamiento", idAlojamiento)
        .param(name: "fechaInicio", fechaInicio)
        .param(name: "fechaFin", fechaFin)
        .param(name: "nHuespedes", nHuespedes)
        .param(name: "tipoPago", tipoPago)
        .query(mappedClass: Integer.class)
        .single();
}
```

Repository/ReservaRepository.java

cancelarReserva

```java
    // Cancelar reserva
    public Integer cancelarReserva(Integer idReserva) {
        String sql = "CALL cancelarReserva(:idReserva, NULL)";

        return jdbcClient.sql(sql)
            .param(name: "idReserva", idReserva)
            .query(mappedClass: Integer.class)
            .single();
    }
}
```

Triggers

TRIGGER 1

Validar la reserva

Impide confirmar una reserva si:

Las fechas se traslapan con otra confirmada

Los huéspedes superan la capacidad

No se cumple la estadía mínima

TRIGGER 2

Auditar los cambios

Registra cada modificación de:

El precio por noche

La política de cancelación

Guarda el valor anterior, el nuevo, el usuario y la fecha

Trigger 1: Evitar reservas dobles

fn_validar_reserva y trg_validar_reserva

FUNCIÓN

fn_validar_reserva

```
CREATE OR REPLACE FUNCTION fn_validar_reserva()
RETURNS TRIGGER AS $$
DECLARE
    v_capacidad INT;
    v_estadia_min INT;
    v_traslapes INT;
BEGIN
    IF NEW.estado_reserva = 'CONFIRMADA' THEN

        -- 1. VALIDAR CAPACIDAD Y ESTADÍA MÍNIMA DEL ALOJAMIENTO
        SELECT capacidad, estadia_min
        INTO v_capacidad, v_estadia_min
        FROM alojamiento
        WHERE id_alojamiento = NEW.id_alojamiento;

        IF NEW.n_huespedes > v_capacidad THEN
            RAISE EXCEPTION 'La cantidad de huéspedes (%) supera la capacidad máxima de alojamiento permitido (%).',
                            NEW.n_huespedes, v_capacidad;
        END IF;

        IF (NEW.fecha_fin - NEW.fecha_inicio) < v_estadia_min THEN
            RAISE EXCEPTION 'La estadía seleccionada (%) es menor que la estadía mínima (%) requerida.',
                            (NEW.fecha_fin - NEW.fecha_inicio), v_estadia_min;
        END IF;

        -- 2. VALIDAR TRASLAPE DE FECHAS
        SELECT COUNT(*) INTO v_traslapes
        FROM reserva
        WHERE id_alojamiento = NEW.id_alojamiento
          AND estado_reserva = 'CONFIRMADA'
          AND (NEW.id_reserva IS NULL OR id_reserva <> NEW.id_reserva)
          AND NEW.fecha_inicio < fecha_fin
          AND NEW.fecha_fin > fecha_inicio;

        IF v_traslapes > 0 THEN
            RAISE EXCEPTION 'Las fechas seleccionadas se traslapan con otra reservación confirmada para este alojamiento.';
        END IF;

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql^;
```

TRIGGER

trg_validar_reserva

```
CREATE OR REPLACE TRIGGER trg_validar_reserva
BEFORE INSERT OR UPDATE ON reserva
FOR EACH ROW
EXECUTE FUNCTION fn_validar_reserva()^;
```

Trigger 2: Auditoría

fn_auditar_alojamiento y trg_auditoria_alojamiento

FUNCIÓN

fn_auditar_alojamiento

```
CREATE OR REPLACE FUNCTION fn_auditar_alojamiento()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.precio_noche IS DISTINCT FROM OLD.precio_noche THEN
        INSERT INTO auditoria (
            id_alojamiento, campo_cambiado, valor_anterior,
            valor_nuevo, usuario_cambio, fecha_cambio
        )
        VALUES (
            OLD.id_alojamiento,
            'precio_noche',
            OLD.precio_noche::VARCHAR,
            NEW.precio_noche::VARCHAR,
            COALESCE(NULLIF(current_setting('app.usuario', true), ''), 'SISTEMA'),
            NOW()
        );
    END IF;

    IF NEW.politica_cancel IS DISTINCT FROM OLD.politica_cancel THEN
        INSERT INTO auditoria (
            id_alojamiento, campo_cambiado, valor_anterior,
            valor_nuevo, usuario_cambio, fecha_cambio
        )
        VALUES (
            OLD.id_alojamiento,
            'politica_cancel',
            OLD.politica_cancel,
            NEW.politica_cancel,
            COALESCE(NULLIF(current_setting('app.usuario', true), ''), 'SISTEMA'),
            NOW()
        );
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql^;
```

TRIGGER

trg_auditoria_alojamiento

```
CREATE OR REPLACE TRIGGER trg_auditoria_alojamiento
BEFORE UPDATE ON alojamiento
FOR EACH ROW
EXECUTE FUNCTION fn_auditar_alojamiento()^;
```

Vista materializada

mv_ocupacion_ingresos e índices

VISTA MATERIALIZADA

mv_ocupacion_ingresos

```
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_ocupacion_ingresos AS
SELECT
    a.ciudad,
    a.tipo_alojamiento,
    EXTRACT(YEAR FROM r.fecha_inicio)::INT  AS anio,
    EXTRACT(MONTH FROM r.fecha_inicio)::INT AS mes,
    COUNT(r.id_reserva)::INT                AS total_reservas,
    COALESCE(SUM(r.precio_total), 0)        AS ingresos_totales
FROM reserva r
JOIN alojamiento a ON a.id_alojamiento = r.id_alojamiento
WHERE r.estado_reserva IN ('CONFIRMADA', 'FINALIZADA')
GROUP BY a.ciudad, a.tipo_alojamiento, anio, mes^;
```

ÍNDICES

3 índices en la base

```
CREATE INDEX IF NOT EXISTS idx_reserva_rango_fechas
    ON reserva USING GIST (daterange(fecha_inicio, fecha_fin))^;
CREATE INDEX IF NOT EXISTS idx_reserva_id_alojamiento ON reserva (id_alojamiento)^;
CREATE INDEX IF NOT EXISTS idx_alojamiento_ciudad ON alojamiento (ciudad)^;
```

Búsqueda y reporte

Fragmentos de código · índice GIST para la búsqueda y vista materializada para el reporte

BÚSQUEDA CON ÍNDICE GIST (AlojamientoRepository)

findDisponibles

```
// Obtener alojamientos disponibles validando fechas con GIST
    public List<AlojamientoEntity> findDisponibles(String ciudad, java.time.LocalDate fechaInicio,
java.time.LocalDate fechaFin, Integer huespedes) {
        // Consulta SQL para obtener alojamientos disponibles en la ciudad, fechas y cantidad de huéspedes
especificados
        // Con el operador && hacemos el llamado a GIST para validar que no se solapen las fechas de reserva con
las fechas de búsqueda
        String sql = """
            SELECT a.* FROM alojamiento a
            WHERE a.ciudad = :ciudad
              AND a.capacidad >= :huespedes
              AND a.id_alojamiento NOT IN (
                  SELECT r.id_alojamiento FROM reserva r
                  WHERE r.estado_reserva IN ('CONFIRMADA', 'FINALIZADA')
                  AND daterange(r.fecha_inicio, r.fecha_fin) && daterange(CAST(:fechaInicio AS date),
CAST(:fechaFin AS date))
              )
        """;
        return jdbcClient.sql(sql)
                .param("ciudad", ciudad)
                .param("huespedes", huespedes)
                .param("fechaInicio", fechaInicio)
                .param("fechaFin", fechaFin)
                .query(alojamientoRowMapper)
                .list();
    }
```

REPORTE · VISTA MATERIALIZADA

ReporteRepository

```
public List<ReporteOcupacionDTO> getReporte() {
        String sql = "SELECT * FROM mv_ocupacion_ingresos ORDER BY anio DESC, mes DESC";
        return jdbcClient.sql(sql).query(mapper).list();
    }
    // Refrescar la vista materializada
    public void refrescarVista() {
        jdbcClient.sql("REFRESH MATERIALIZED VIEW mv_ocupacion_ingresos").update();
    }
```

ReporteService

```
    // Refresca la vista todos los días a las 2:00 AM
    @Scheduled(cron = "0 0 2 * * ?")
    public void refrescarReporteDiario() {
        reporteRepository.refrescarVista();
    }
}
```

Contribuciones

Qué programó o diseñó cada integrante del grupo

INTEGRANTE	TAREAS
Diego Cáceres	Backend , Frontend , Procedimiento almacenado 1 , Trigger 1 y vista materializada
Dania Garrido	Backend, Procedimiento almacenado 1, Trigger 1 e índices
Diego Lefiman	Backend , Procedimiento almacenado 2 y Trigger 2
Isidora Toledo	Backend, archivos de creación de las tablas y llenado de datos y presentación
Matias Urbano	Backend, Frontend, docker, keycloak y conexión con back y front

Conclusiones

01

Desafío técnico

Trabajar sin ORM

Escribir cada consulta SQL y mapear las filas a mano con JDBC costó más, pero dio control

02

Lo que aprendimos

Reglas, capas y seguridad

Las reglas críticas van en PostgreSQL, el código en capas y la API protegida con JWT

03

Por mejorar

Lo que quedó pendiente

- Pantalla de reseñas en el frontend
- Cambiar la estética de la pantalla de iniciar sesión

Obligarnos a no usar ORMs y llevar la lógica compleja directamente a PostgreSQL (con Triggers y Stored Procedures) nos demostró que podemos construir sistemas mucho más rápidos y a prueba de fallos de concurrencia. Separar bien las capas y proteger todo con JWT resultó clave para que la plataforma funcione de forma segura.

